

第38章 基于 SD 卡的 FatFs 文件系统

上一章我们已经全面介绍了 SD 卡的识别和简单的数据读写，也进行了简单的读写测试，不过像这样直接操作 SD 卡存储单元，在实际应用中是不现实的。SD 卡一般用来存放文件，所以都需要加载文件系统到里面。类似于串行 Flash 芯片，我们移植 FatFs 文件系统到 SD 卡内。

对于 FatFs 文件系统的介绍和具体移植过程参考“基于串行 Flash 的 FatFs 文件系统”，这里就不做过多介绍，重点放在 SD 卡与 FatFs 接口函数编写上。与串行 Flash 的 FatFs 文件系统移植例程相比，FatFs 文件系统部分的代码只有 diskio.c 文件有所不同，其他的不用修改，所以一个简易的移植方法是利用原来工程进行修改。下面讲解利用原来工程实现 SD 卡的 FatFs 文件系统。

38.1 FatFs 移植步骤

上一章我们已经完成了 SD 卡驱动程序以及进行了简单的读写测试。该工程有很多东西是现在可以使用的，所以我们先把上一章的工程文件完整的拷贝一份，并修改文件夹名为“SDIO-FatFs 移植与读写测试”，如果此时使用 KEIL 软件打开该工程，应该是编译无错误并实现上一章的测试功能。

接下来，我们到串行 Flash 文件系统移植工程文件的“\SPI—FatFs 移植与读写测试\User”文件夹下拷贝“FATFS”整个文件夹到现在工程文件的“\SDIO—FatFs 移植与读写测试\User”文件夹下，如图 38-1。该文件夹是 FatFs 文件系统的所有代码文件，在串行 Flash 移植 FatFs 文件系统时我们对部分文件做了修改，这里主要是想要保留之前的配置，而不是使用 FatFs 官方源码还需要重新配置。

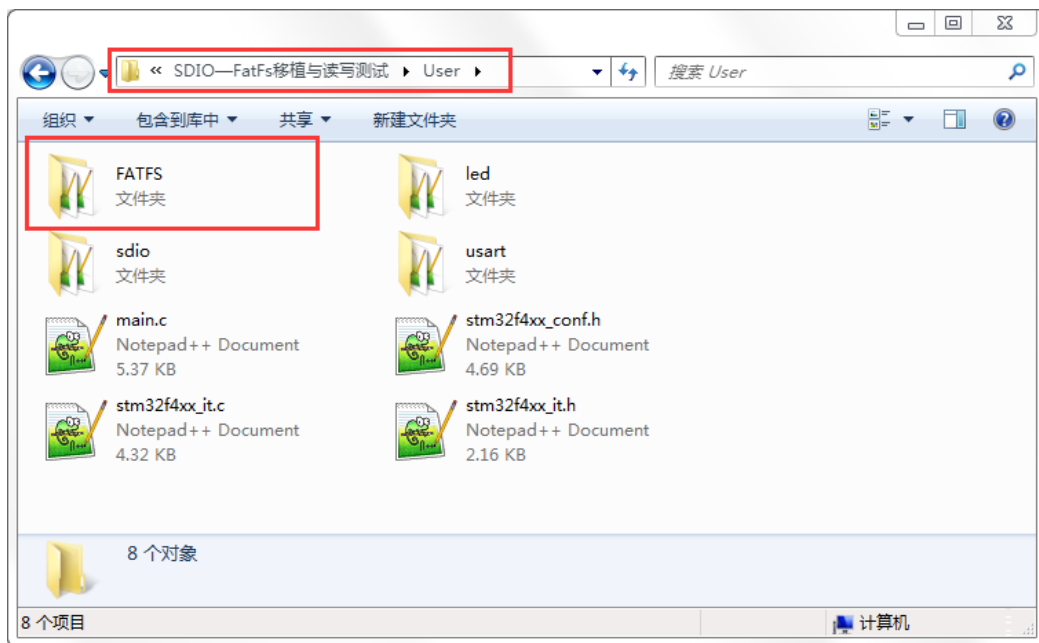


图 38-1 拷贝 FatFs 文件夹

现在就可以使用 KEIL 软件打开“SDIO-FatFs 移植与读写测试”工程文件，并把 FatFs 相关文件添加到工程内，同时把 sdio_test.c 文件移除，参考图 38-2。

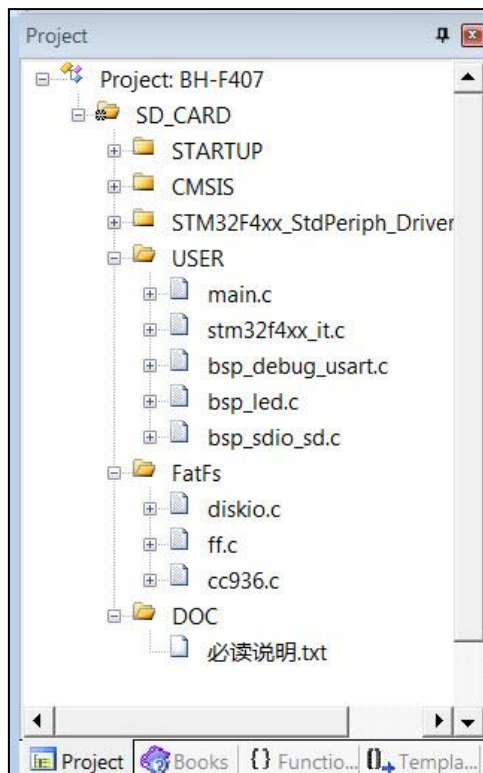


图 38-2 FatFs 工程文件结构

添加文件之后还必须打开工程选项对话框添加相关路径，参考图 38-3。

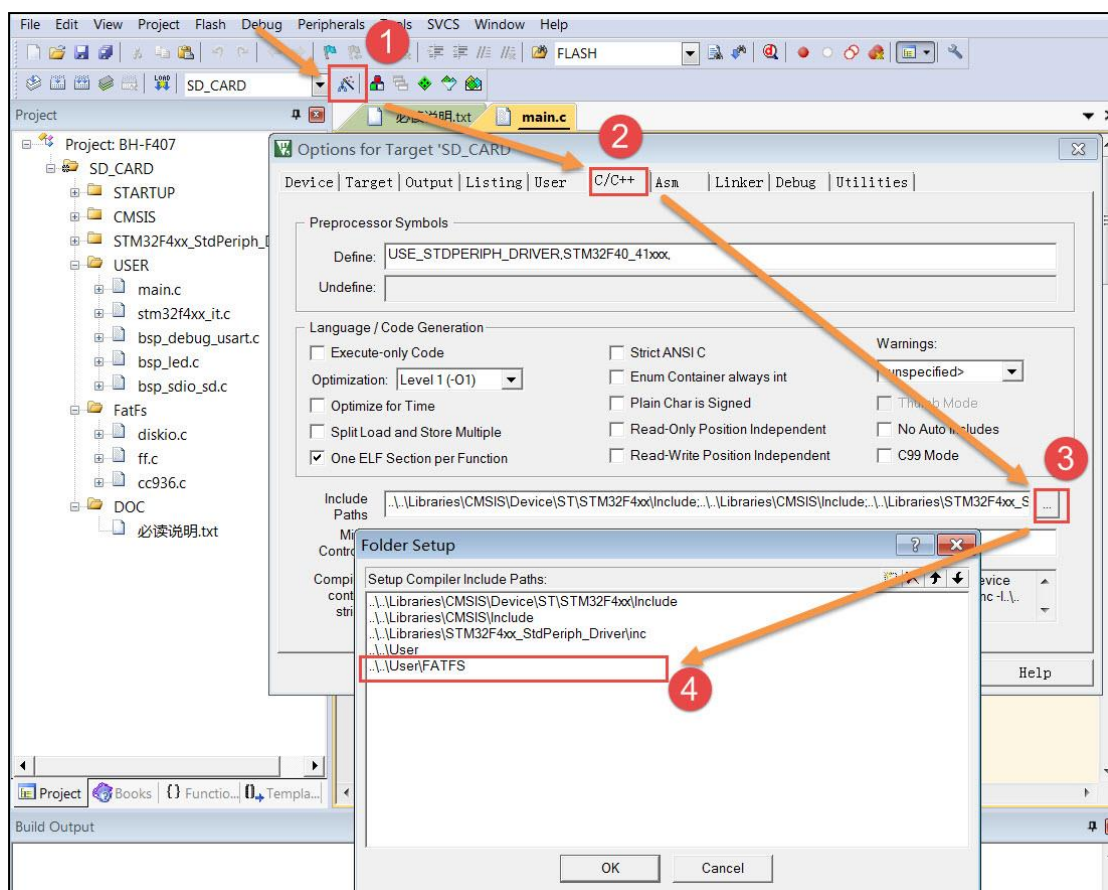


图 38-3 添加 FatFs 路径到工程

操作到这里，工程文件结构就算完整了，接下来就是修改文件代码了。这来有两个文件需要修改，为 diskio.c 文件和 main.c 文件。main.c 文件内容可以参考“SPI—FatFs 移植与读写测试”工程中的 main.c 文件，只有做小细节修改而已。这来重点讲解 diskio.c 文件，也是整个移植的重点。

38.2 FatFs 接口函数

FatFs 文件系统与存储设备的连接函数在 diskio.c 文件中，主要有 5 个函数需要我们编写的。

宏定义和存储设备状态获取函数

代码清单 38-1 宏定义和 disk_status 函数

```
1 //宏定义
2 #define ATA                0        // SD 卡
3 #define SPI_FLASH          1        // 预留外部 SPI Flash 使用
4 // SD 卡块大小
5 #define SD_BLOCKSIZE      512
6
7 //存储设备状态获取
8 DSTATUS disk_status (
9     BYTE pdrv /* 物理编号 */
10 )
11 {
12     DSTATUS status = STA_NOINIT;
13     switch (pdrv) {
14         case ATA: /* SD CARD */
```

```
15     status &= ~STA_NOINIT;
16     break;
17
18     case SPI_FLASH:          /* SPI Flash */
19         break;
20
21     default:
22         status = STA_NOINIT;
23 }
24 return status;
25 }
```

FatFs 支持同时挂载多个存储设备，通过定义为不同编号以区别。SD 卡一般定义为编号 0，编号 1 预留给串行 Flash 芯片使用。使用宏定义方式给出 SD 卡块大小，方便修改。实际上，SD 卡块大小一般都是设置为 512 字节的，不管是标准 SD 卡还是高容量 SD 卡。

disk_status 函数要求返回存储设备的当前状态，对于 SD 卡一般返回 SD 卡插入状态，这里直接返回正常状态。

存储设备初始化函数

代码清单 38-2 disk_initialize 函数

```
1 DSTATUS disk_initialize (
2     BYTE pdrv          /* 物理编号 */
3 )
4 {
5     DSTATUS status = STA_NOINIT;
6     switch (pdrv) {
7     case ATA:           /* SD CARD */
8         if (SD_Init() == SD_OK) {
9             status &= ~STA_NOINIT;
10        } else {
11            status = STA_NOINIT;
12        }
13        break;
14
15     case SPI_FLASH:     /* SPI Flash */
16         break;
17
18     default:
19         status = STA_NOINIT;
20    }
21    return status;
22 }
23 }
```

该函数用于初始化存储设备，一般包括相关 GPIO 初始化、外设环境初始化、中断配置等等。对于 SD 卡，直接调用 SD_Init 函数实现对 SD 卡初始化，如果函数返回 SD_OK 说明 SD 卡正确插入，并且控制器可以与之正常通信。

存储设备数据读取函数

代码清单 38-3 disk_read 函数

```
1 DRESULT disk_read (
2     BYTE pdrv,          /* 设备物理编号(0..) */
3     BYTE *buff,         /* 数据缓存区 */
4     DWORD sector,       /* 扇区首地址 */
5     UINT count          /* 扇区个数(1..128) */
6 )
7 {
8     DRESULT status = RES_PARERR;
9     SD_Error SD_state = SD_OK;
10
11     switch (pdrv) {
```

```
12     case ATA: /* SD CARD */
13         if ((DWORD)buff&3) {
14             DRESULT res = RES_OK;
15             DWORD scratch[SD_BLOCKSIZE / 4];
16
17             while (count-->0) {
18                 res = disk_read(ATA, (void *)scratch, sector++, 1);
19
20                 if (res != RES_OK) {
21                     break;
22                 }
23                 memcpy(buff, scratch, SD_BLOCKSIZE);
24                 buff += SD_BLOCKSIZE;
25             }
26             return res;
27         }
28
29         SD_state=SD_ReadMultiBlocks(buff,sector*SD_BLOCKSIZE,
30                                     SD_BLOCKSIZE,count);
31         if (SD_state==SD_OK) {
32             /* Check if the Transfer is finished */
33             SD_state=SD_WaitReadOperation();
34             while (SD_GetStatus() != SD_TRANSFER_OK);
35         }
36         if (SD_state!=SD_OK)
37             status = RES_PARERR;
38         else
39             status = RES_OK;
40         break;
41
42     case SPI_FLASH:
43         break;
44
45     default:
46         status = RES_PARERR;
47     }
48     return status;
49 }
```

disk_read 函数用于从存储设备指定地址开始读取一定的数量的数据到指定存储区内。对于 SD 卡，最重要是使用 SD_ReadMultiBlocks 函数读取多块数据到存储区。这里需要注意的地方是 SD 卡数据操作是使用 DMA 传输的，并设置数据尺寸为 32 位大小，为实现数据正确传输，要求存储区是 4 字节对齐。在某些情况下，FatFs 提供的 buff 地址不是 4 字节对齐，这会导致 DMA 数据传输失败，所以为保证数据传输正确，可以先判断存储区地址是否是 4 字节对齐，如果存储区地址已经是 4 字节对齐，无需其他处理，直接使用 SD_ReadMultiBlocks 函数执行多块读取即可。如果判断得到地址不是 4 字节对齐，则先申请一个 4 字节对齐的临时缓冲区，即局部数组变量 scratch，通过定义为 DWORD 类型可以使得其自动 4 字节对齐，scratch 所占的总存储空间也是一个块大小，这样把一个块数据读取到 scratch 内，然后把 scratch 存储器内容拷贝到 buff 地址空间上就可以了。

SD_ReadMultiBlocks 函数用于从 SD 卡内读取多个块数据，它有四个形参，分别为存储区地址指针、起始块地址、块大小以及块数量。为保证数据传输完整，还需要调用 SD_WaitReadOperation 函数和 SD_GetStatus 函数检测和保证传输完成。

存储设备数据写入函数

代码清单 38-4 disk_write 函数

```
1 #if _USE_WRITE
2 DRESULT disk_write (
3     BYTE pdrv,          /* 设备物理编号(0..) */
```

```
4     const BYTE *buff, /* 欲写入数据的缓存区 */
5     DWORD sector,      /* 扇区首地址 */
6     UINT count         /* 扇区个数(1..128) */
7 )
8 {
9     DRESULT status = RES_PARERR;
10    SD_Error SD_state = SD_OK;
11
12    if (!count) {
13        return RES_PARERR;    /* Check parameter */
14    }
15
16    switch (pdrv) {
17    case ATA: /* SD CARD */
18        if ((DWORD)buff&3) {
19            DRESULT res = RES_OK;
20            DWORD scratch[SD_BLOCKSIZE / 4];
21
22            while (count--) {
23                memcpy(scratch, buff, SD_BLOCKSIZE);
24                res = disk_write(ATA, (void *)scratch, sector++, 1);
25                if (res != RES_OK) {
26                    break;
27                }
28                buff += SD_BLOCKSIZE;
29            }
30            return res;
31        }
32
33        SD_state=SD_WriteMultiBlocks((uint8_t *)buff, sector*SD_BLOCKSIZE,
34                                     SD_BLOCKSIZE, count);
35
36        if (SD_state==SD_OK) {
37            /* Check if the Transfer is finished */
38            SD_state=SD_WaitReadOperation();
39
40            /* Wait until end of DMA transfer */
41            while (SD_GetStatus() != SD_TRANSFER_OK);
42        }
43        if (SD_state!=SD_OK)
44            status = RES_PARERR;
45        else
46            status = RES_OK;
47        break;
48    case SPI_FLASH:
49        break;
50
51    default:
52        status = RES_PARERR;
53    }
54    return status;
55 }
56 #endif
```

`disk_write` 函数用于向存储设备指定地址写入指定数量的数据。对于 SD 卡，执行过程与 `disk_read` 函数是非常相似，也必须先检测存储区地址是否是 4 字节对齐，如果是 4 字节对齐则直接调用 `SD_WriteMultiBlocks` 函数完成多块数据写入操作。如果不是 4 字节对齐，申请一个 4 字节对齐的临时缓冲区，先把待写入的数据拷贝到该临时缓冲区内，然后才写入到 SD 卡。

`SD_WriteMultiBlocks` 函数是向 SD 卡写入多个块数据，它有四个形参，分别为存储区地址指针、起始块地址、块大小以及块数量，它与 `SD_ReadMultiBlocks` 函数执行相互过程。最后也是需要使用相关函数保存数据写入完整才退出 `disk_write` 函数。

其他控制函数

代码清单 38-5 disk_ioctl 函数

```
1 #if _USE_IOCTL
2 DRESULT disk_ioctl (
3     BYTE pdrv,      /* 物理编号 */
4     BYTE cmd,        /* 控制指令 */
5     void *buff       /* 写入或者读取数据地址指针 */
6 )
7 {
8     DRESULT status = RES_PARERR;
9     switch (pdrv) {
10     case ATA: /* SD CARD */
11         switch (cmd) {
12             // Get R/W sector size (WORD)
13             case GET_SECTOR_SIZE :
14                 *(WORD *)buff = SD_BLOCKSIZE;
15                 break;
16             // Get erase block size in unit of sector (DWORD)
17             case GET_BLOCK_SIZE :
18                 *(DWORD *)buff = SDCardInfo.CardBlockSize;
19                 break;
20
21             case GET_SECTOR_COUNT:
22                 *(DWORD*)buff = SDCardInfo.CardCapacity/SDCardInfo.CardBlockSize;
23                 break;
24             case CTRL_SYNC :
25                 break;
26         }
27         status = RES_OK;
28         break;
29
30     case SPI_FLASH:
31         break;
32
33     default:
34         status = RES_PARERR;
35     }
36     return status;
37 }
38 #endif
```

disk_ioctl 函数有三个形参，pdrv 为设备物理编号，cmd 为控制指令，包括发出同步信号、获取扇区数目、获取扇区大小、获取擦除块数量等等指令，buff 为指令对应的数据指针。

对于 SD 卡，为支持格式化功能，需要用到获取扇区数量(GET_SECTOR_COUNT)指令和获取块尺寸(GET_BLOCK_SIZE)。另外，SD 卡扇区大小为 512 字节，串行 Flash 芯片一般设置扇区大小为 4096 字节，所以需要用到获取扇区大小(GET_SECTOR_SIZE)指令。

至此，基于 SD 卡的 FatFs 文件系统移植就已经完成了，最重要就是 diskio.c 文件中 5 个函数的编写。接下来就编写 FatFs 基本的文件操作检测移植代码是否可以正确执行。

38.3 FatFs 功能测试

主要的测试包括格式化测试、文件写入测试和文件读取测试三个部分，主要程序都在 main.c 文件中实现。

变量定义

代码清单 38-6 变量定义


```
1 FATFS fs; /* FatFs 文件系统对象 */
2 FIL fnew; /* 文件对象 */
3 FRESULT res_sd; /* 文件操作结果 */
4 UINT fnum; /* 文件成功读写数量 */
5 BYTE ReadBuffer[1024]= {0}; /* 读缓冲区 */
6 BYTE WriteBuffer[] = /* 写缓冲区 */
7 "欢迎使用野火 STM32 F407 开发板 今天是个好日子, 新建文件系统测试文件\r\n";
```

FATFS 是在 ff.h 文件定义的一个结构体类型, 针对的对象是物理设备, 包含了物理设备的物理编号、扇区大小等信息, 一般我们都需要为每个物理设备定义一个 FATFS 变量。

FIL 也是在 ff.h 文件定义的一个结构体类型, 针对的对象是文件系统内具体的文件, 包含了文件很多基本属性, 比如文件大小、路径、当前读写地址等等。如果需要在同一时间打开多个文件进行读写, 才需要定义多个 FIL 变量, 不然一般定义一个 FIL 变量即可。

FRESULT 是也在 ff.h 文件定义的一个枚举类型, 作为 FatFs 函数的返回值类型, 主要管理 FatFs 运行中出现的错误。总共有 19 种错误类型, 包括物理设备读写错误、找不到文件、没有挂载工作空间等等错误。这在实际编程中非常重要, 当有错误出现是我们要停止文件读写, 通过返回值我们可以快速定位到错误发生的可能地点。如果运行没有错误才返回 FR_OK。

fnum 是个 32 位无符号整形变量, 用来记录实际读取或者写入数据的数组。

buffer 和 textFileBuffer 分别对应读取和写入数据缓存区, 都是 8 位无符号整形数组。

主函数

代码清单 38-7 main 函数

```
1 int main(void)
2 {
3
4
5
6     /* 初始化 LED */
7     LED_GPIO_Config();
8     LED_BLUE;
9
10    /* 初始化调试串口, 一般为串口 1 */
11    Debug_USART_Config();
12    printf("\r\n***** 这是一个 SD 卡 文件系统实验 *****\r\n");
13
14    //在外部 SPI Flash 挂载文件系统, 文件系统挂载时会对 SPI 设备初始化
15    res_sd = f_mount(&fs, "0:", 1);
16
17    /*----- 格式化测试 -----*/
18    /* 如果没有文件系统就格式化创建创建文件系统 */
19    if (res_sd == FR_NO_FILESYSTEM) {
20        printf("》SD 卡还没有文件系统, 即将进行格式化...\r\n");
21        /* 格式化 */
22        res_sd=f_mkfs("0:", 0, 0);
23
24        if (res_sd == FR_OK) {
25            printf("》SD 卡已成功格式化文件系统.\r\n");
26            /* 格式化后, 先取消挂载 */
27            res_sd = f_mount(NULL, "0:", 1);
28            /* 重新挂载 */
29            res_sd = f_mount(&fs, "0:", 1);
30        } else {
31            LED_RED;
32            printf("《《格式化失败。》》\r\n");
33            while (1);
34        }
35    } else if (res_sd!=FR_OK) {
```



```
36     printf("!! SD 卡挂载文件系统失败。(d)\r\n",res_sd);
37     printf("!! 可能原因: SD 卡初始化不成功.\r\n");
38     while (1);
39 } else {
40     printf("》文件系统挂载成功, 可以进行读写测试\r\n");
41 }
42
43 /*----- 文件系统测试: 写测试 -----*/
44 /* 打开文件, 如果文件不存在则创建它 */
45 printf("\r\n***** 即将进行文件写入测试... *****\r\n");
46 res_sd=f_open(&fnew,"0:FatFs 读写测试文件.txt",FA_CREATE_ALWAYS|FA_WRITE);
47 if ( res_sd == FR_OK ) {
48     printf("》打开/创建 FatFs 读写测试文件.txt 文件成功, 向文件写入数据.\r\n");
49     /* 将指定存储区内容写入到文件内 */
50     res_sd=f_write(&fnew,WriteBuffer,sizeof(WriteBuffer),&fnum);
51     if (res_sd==FR_OK) {
52         printf("》文件写入成功, 写入字节数据: %d\r\n",fnum);
53         printf("》向文件写入的数据为: \r\n%s\r\n",WriteBuffer);
54     } else {
55         printf("!! 文件写入失败: (d)\n",res_sd);
56     }
57     /* 不再读写, 关闭文件 */
58     f_close(&fnew);
59 } else {
60     LED_RED;
61     printf("!! 打开/创建文件失败.\r\n");
62 }
63
64 /*----- 文件系统测试: 读测试 -----*/
65 printf("***** 即将进行文件读取测试... *****\r\n");
66 res_sd=f_open(&fnew,"0:FatFs 读写测试文件.txt",FA_OPEN_EXISTING|FA_READ);
67 if (res_sd == FR_OK) {
68     LED_GREEN;
69     printf("》打开文件成功.\r\n");
70     res_sd = f_read(&fnew, ReadBuffer, sizeof(ReadBuffer), &fnum);
71     if (res_sd==FR_OK) {
72         printf("》文件读取成功, 读到字节数据: %d\r\n",fnum);
73         printf("》读取得的文件数据为: \r\n%s \r\n", ReadBuffer);
74     } else {
75         printf("!! 文件读取失败: (d)\n",res_sd);
76     }
77 } else {
78     LED_RED;
79     printf("!! 打开文件失败.\r\n");
80 }
81 /* 不再读写, 关闭文件 */
82 f_close(&fnew);
83
84 /* 不再使用文件系统, 取消挂载文件系统 */
85 f_mount(NULL,"0:",1);
86
87 /* 操作完成, 停机 */
88 while (1) {
89 }
90 }
```

程序中首先初始化 RGB 彩灯和调试串口, 用来指示程序进程。

FatFs 的第一步工作就是使用 f_mount 函数挂载工作区。f_mount 函数有三个形参, 第一个参数是指向 FATFS 变量指针, 如果赋值为 NULL 可以取消物理设备挂载。第二个参数为逻辑设备编号, 使用设备根路径表示, 与物理设备编号挂钩, 在代码清单 38-1 中我们

定义 SD 卡物理编号为 0，所以这里使用“0:”。第三个参数可选 0 或 1，1 表示立即挂载，0 表示不立即挂载，延迟挂载。f_mount 函数会返回一个 FRESULT 类型值，指示运行情况。

如果 f_mount 函数返回值为 FR_NO_FILESYSTEM，说明 SD 卡没有 FAT 文件系统。我们就必须对 SD 卡进行格式化处理。使用 f_mkfs 函数可以实现格式化操作。f_mkfs 函数有三个形参，第一个参数为逻辑设备编号；第二参数可选 0 或者 1，0 表示设备为一般硬盘，1 表示设备为软盘。第三个参数指定扇区大小，如果为 0，表示通过代码清单 38-5 中 disk_ioctl 函数获取。格式化成功后需要先取消挂载原来设备，再重新挂载设备。

在设备正常挂载后，就可以进行文件读写操作了。使用文件之前，必须使用 f_open 函数打开文件，不再使用文件必须使用 f_close 函数关闭文件，这个跟电脑端操作文件步骤类似。f_open 函数有三个形参，第一个参数为文件对象指针。第二参数为目标文件，包含绝对路径的文件名称和后缀名。第三个参数为访问文件模式选择，可以是打开已经存在的文件模式、读模式、写模式、新建模式、总是新建模式等的或运行结果。比如对于写测试，使用 FA_CREATE_ALWAYS 和 FA_WRITE 组合模式，就是总是新建文件并进行写模式。

f_close 函数用于不再对文件进行读写操作关闭文件，f_close 函数只要一个形参，为文件对象指针。f_close 函数运行可以确保缓冲区完全写入到文件内。

成功打开文件之后就可以使用 f_write 函数和 f_read 函数对文件进行写操作和读操作。这两个函数用到的参数是一致的，只不过一个是数据写入，一个是数据读取。f_write 函数第一个形参为文件对象指针，使用与 f_open 函数一致即可。第二个参数为待写入数据的首地址，对于 f_read 函数就是用来存放读出数据的首地址。第三个参数为写入数据的字节数，对于 f_read 函数就是欲读取数据的字节数。第四个参数为 32 位无符号整形指针，这里使用 fnum 变量地址赋值给它，在运行读写操作函数后，fnum 变量指示成功读取或者写入的字节个数。

最后，不再使用文件系统时，使用 f_mount 函数取消挂载。

下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。程序开始运行后，RGB 彩灯为蓝色，在串口调试助手可看到格式化测试、写文件检测和读文件检测三个过程；最后如果所有读写操作都正常，RGB 彩灯会指示为绿色，如果在运行中 FatFs 出现错误 RGB 彩灯指示为红色。正确执行例程程序后可以使用读卡器将 SD 卡在电脑端打开，我们可以在 SD 卡根目录下看到“FatFs 读写测试文件.txt”文件，这与程序设计是相吻合的。

